CS1103 Final Project

Guilherme Machado da Silva

Abstract

In this project, I built a Library Management System. The goal was to take a database concept on paper and turn it into an application. I started by drawing an ER diagram to plan the structure and then brought it to life in SQLite using SQL commands. To interact with the database, I wrote a Java app using JDBC. This app lets users add, view, update, and delete library records, and it even uses manual database transactions to make sure two people can't check out the same book at once. Overall, this project shows the entire step-by-step process of building a backend database and connecting it to a real-world program.

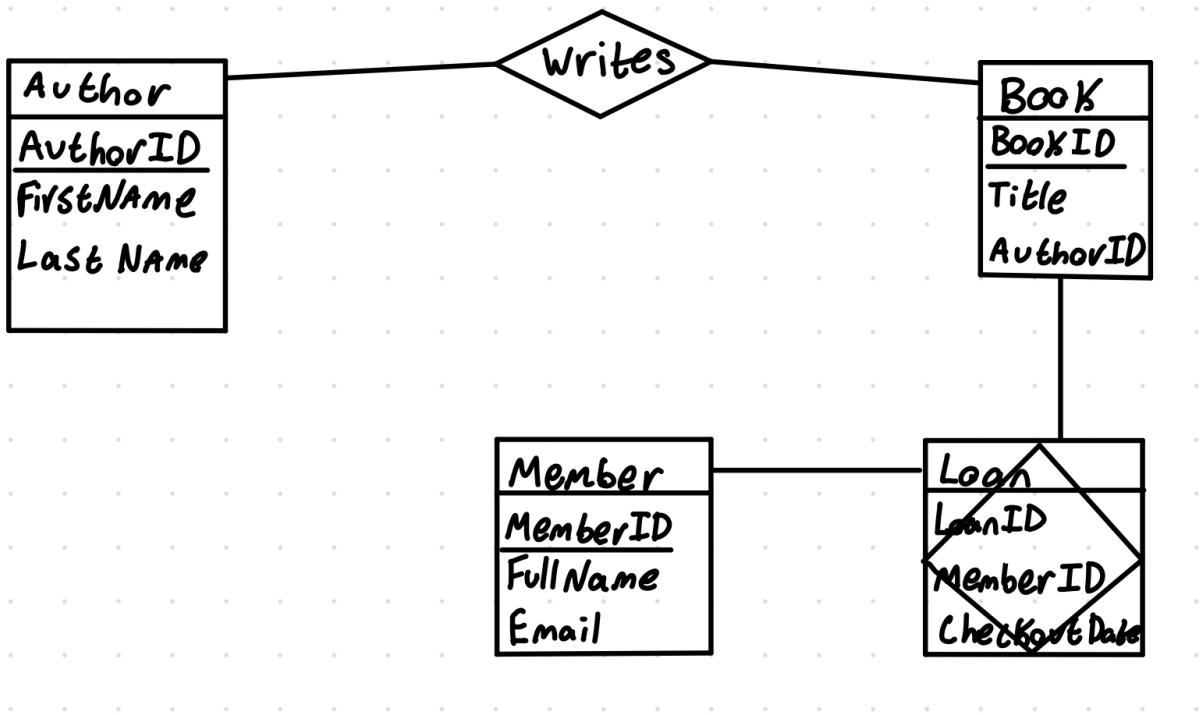
Introduction

Libraries deal with a lot of data every day, they keep track of new books and manage who checked out what. Doing this by hand is really messy and it leads to lost books and disorganized records. The goal for this project was to solve that problem by building a database system for a library.

This project is about learning how the development works. I started with an ER diagram, and I turned that into an SQLite database using some DDL and DML statements, and I also built a Java app to control it. Using JDBC, my Java program connects directly to the database, giving the user a simple menu to run everyday library tasks like adding new members, returning books, and managing the library's inventory.

Database Design

I designed the database around 3 entities: the Author, the Book, and the Member. To make sure everything is organized they all get their own ID which is their primary key. When I wrote the SQL code to build the tables, I linked the Author to the Book by putting the AuthorID inside the book table as a Foreign Key. The Loan entity is represented as an associative entity in the ER diagram. In the database it is a bridge that connects a member to the Book that was checked out.



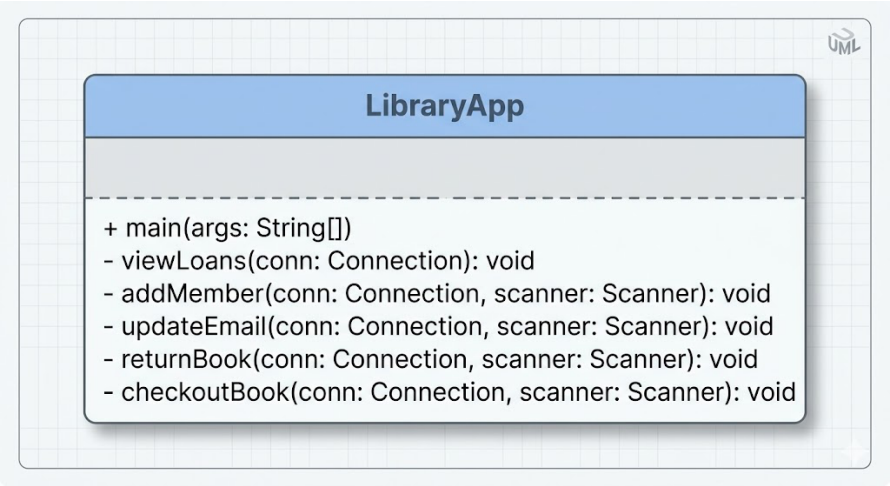
Implementation

To bring the design to life, I wrote a Java program called LibraryDataInserter that uses JDBC to create the library.db SQLite file, run the create table statements, and insert some sample data. With the database built I created the SQL queries for some standard CRUD operations to manage the data. My application uses SELECT to display checked-out books, INSERT to add new members, UPDATE to change a member's email, and DELETE to remove a loan when the book is returned.

```java
// Create  Entity Tables
String createAuthor = "CREATE TABLE IF NOT EXISTS Author (" +
    "AuthorID VARCHAR(6) PRIMARY KEY, " +
    "FirstName VARCHAR(50) NOT NULL, " +
    "LastName VARCHAR(50) NOT NULL)";

stmt.executeUpdate(createAuthor);

String createMember = "CREATE TABLE IF NOT EXISTS Member (" +
    "MemberID VARCHAR(5) PRIMARY KEY, " +
    "FullName VARCHAR(100) NOT NULL, " +
    "Email VARCHAR(100) UNIQUE)";

stmt.executeUpdate(createMember);

String createBook = "CREATE TABLE IF NOT EXISTS Book (" +
    "BookID VARCHAR(5) PRIMARY KEY, " +
    "Title VARCHAR(200) NOT NULL, " +
    "AuthorID VARCHAR(6), " +
    "FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID))";

stmt.executeUpdate(createBook);

// Create Relation Table
String createLoan = "CREATE TABLE IF NOT EXISTS Loan (" +
    "LoanID VARCHAR(5) PRIMARY KEY, " +
    "BookID VARCHAR(5), " +
    "MemberID VARCHAR(5), " +
    "CheckoutDate DATE NOT NULL, " +
    "FOREIGN KEY (BookID) REFERENCES Book(BookID), " +
    "FOREIGN KEY (MemberID) REFERENCES Member(MemberID))";

stmt.executeUpdate(createLoan);

System.out.println(x: "Tables were created successfully");
System.out.println();
```

Application

To make the database useful, I built a java application using a scanner for input. I also organized the code into some reusable methods like addMember() and returnBook(), which are mapped out in the class diagram below.



Important findings and Challenges

One of the biggest challenges was preventing a "race condition" where two people might try to check out the same book at the exact same time. To solve this, I used conn.setAutoCommit(false) in my Java code to turn the checkout process into a locked transaction. The program first checks if the book is available; if it is, the loan is inserted and permanently saved using a commit(), but if the book is already taken, the entire process is cancelled with a rollback() to maintain strict data integrity.

Conclusion

Building this Library Management System was a very practical experience helped me understand the difference between a theoretical database design and an actual software development. I successfully applied ER modeling, wrote structural DDL and operational DML statements, and I securely connected a Java application to an SQLite database. This project proves how relational databases and programming languages work together to solve real-world administrative problems.

Bibliography

Official Documentation:

Oracle. (n.d.). *Java Platform, Standard Edition (Java SE) Documentation*. Retrieved from https://docs.oracle.com/en/java/

SQLite. (n.d.). *SQLite Official Documentation*. Retrieved from https://www.sqlite.org/docs.html

External Libraries and Drivers:

Taro L. Saito. (n.d.). *SQLite JDBC Driver* [Software]. GitHub. Retrieved from https://github.com/xerial/sqlite-jdbc

Appendix

Database Setup Code:

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.SQLException;

import java.sql.Statement;


// Name: Guilherme Machado da Silva

// ID: 3800746

//

// First Compile in the Terminal then,

// Use this command to execute: java --enable-native-access=ALL-UNNAMED -classpath ".:sqlite-jdbc.jar:slf4j-simple.jar:slf4j-api.jar" LibraryDataInserter
```

```
public class LibraryDataInserter {
  public static void main(String[] args){
    Connection conn = null;

    try {
      // Load the SQLite driver
      Class.forName("org.sqlite.JDBC");

      // Connect to the database
      conn = DriverManager.getConnection("jdbc:sqlite:library.db");

      // Creating a statement
      Statement stmt = conn.createStatement();

      System.out.println("Building database tables...");

      // Create  Entity Tables
      String createAuthor = "CREATE TABLE IF NOT EXISTS Author (" +
        "AuthorID VARCHAR(6) PRIMARY KEY, " +
        "FirstName VARCHAR(50) NOT NULL, " +
        "LastName VARCHAR(50) NOT NULL)";

      stmt.executeUpdate(createAuthor);

      String createMember = "CREATE TABLE IF NOT EXISTS Member (" +
        "MemberID VARCHAR(5) PRIMARY KEY, " +
```

```
    "FullName VARCHAR(100) NOT NULL, " +
    "Email VARCHAR(100) UNIQUE)";

stmt.executeUpdate(createMember);

String createBook = "CREATE TABLE IF NOT EXISTS Book (" +
    "BookID VARCHAR(5) PRIMARY KEY, " +
    "Title VARCHAR(200) NOT NULL, " +
    "AuthorID VARCHAR(6), " +
    "FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID))";

stmt.executeUpdate(createBook);

// Create Relation Table
String createLoan = "CREATE TABLE IF NOT EXISTS Loan (" +
    "LoanID VARCHAR(5) PRIMARY KEY, " +
    "BookID VARCHAR(5), " +
    "MemberID VARCHAR(5), " +
    "CheckoutDate DATE NOT NULL, " +
    "FOREIGN KEY (BookID) REFERENCES Book(BookID), " +
    "FOREIGN KEY (MemberID) REFERENCES Member(MemberID))";

stmt.executeUpdate(createLoan);

System.out.println("Tables were created successfully");
System.out.println();
```

```
        // INSERTING DATA INTO THE DATABASE
        System.out.println("Inserting data into the database");

        // Inserting into the Author Table
        String insertAuthors = "INSERT INTO Author (AuthorID, FirstName, LastName) VALUES " +
            "('00001', 'Lewis', 'Loftus')," +
            "('00002', 'Liana', 'Cincotti')," +
            "('00003', 'Jeff', 'Kinney')";

        stmt.executeUpdate(insertAuthors);

        // Inserting into the Member Table
        String insertMembers = "INSERT INTO Member (MemberID, FullName, Email) VALUES " +
            "('00001', 'Guilherme Machado da Silva', 'gui.silva.machado@unb.ca')," +
            "('00002', 'Meaghan Mahoney', 'meaghan.mahoney@unb.ca')," +
            "('00003', 'Lacey Allen', 'lacey.allen@icloud.com')";

        stmt.executeUpdate(insertMembers);

        // Inserting into the Book Table
        String insertBooks = "INSERT INTO Book (BookID, Title, AuthorID) VALUES " +
            "('00001', 'Java Software Solutions', '00001')," +
            "('00002', 'Picking Daisies on Sundays', '00002')," +
            "('00003', 'Diary of a Wimpy Kid', '00003')";
```

```
        stmt.executeUpdate(insertBooks);

        // Inserting into the Loan Table
        String insertLoans = "INSERT INTO Loan (LoanID, BookID, MemberID, CheckoutDate)
VALUES " +
            "('00001', '00001', '00001', '2026-03-20')," +
            "('00002', '00002', '00001', '2026-03-22')," +
            "('00003', '00003', '00003', '2026-03-24')";

        stmt.executeUpdate(insertLoans);

        System.out.println("All data was inserted sucessfully");
        System.out.println();
    }

    catch (SQLException | ClassNotFoundException e) {
        e.printStackTrace();
    }

    finally {
        try {
            if (conn != null) {
                conn.close();
            }
        }
```

```
        catch (SQLException e) {
            e.printStackTrace();
        }
    }
  }
}
```

Application Code

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

// Name: Guilherme Machado da Silva
// ID: 3800746
//
// Compile: javac LibraryApp.java
// Run: java --enable-native-access=ALL-UNNAMED -classpath ".:sqlite-jdbc.jar:slf4j-simple.jar:slf4j-api.jar" LibraryApp

public class LibraryApp {

  public static void main(String[] args) {
```

```
// Standard local variables
Connection conn = null;
Scanner scanner = new Scanner(System.in);

try {
    Class.forName("org.sqlite.JDBC");
    conn = DriverManager.getConnection("jdbc:sqlite:library.db");

    boolean running = true;

    // The Interactive Menu Loop
    while (running) {
        System.out.println("\n=== LIBRARY MANAGEMENT SYSTEM ===");
        System.out.println("1. View all Checked Out Books");
        System.out.println("2. Add a New Member");
        System.out.println("3. Update a Member's Email");
        System.out.println("4. Checkout a Book");
        System.out.println("5. Return a Book");
        System.out.println("6. Exit Program");
        System.out.print("Please select an option (1-5): ");

        int choice = scanner.nextInt();
        scanner.nextLine(); // Consume the newline
        System.out.println();
```

```
        // Passing the local variables to our functions
        if (choice == 1) {
            viewLoans(conn);
        } else if (choice == 2) {
            addMember(conn, scanner);
        } else if (choice == 3) {
            updateEmail(conn, scanner);
        } else if (choice == 4) {
            checkoutBook(conn, scanner);
        } else if (choice == 5) {
            returnBook(conn, scanner);
        } else if (choice == 6) {
            System.out.println("Exiting the Library System. Goodbye!");
            running = false;
        } else {
            System.out.println("Invalid choice. Please try again.");
        }
    }

} catch (SQLException | ClassNotFoundException e) {
    System.out.println("Database Connection Error!");
    e.printStackTrace();
} finally {
    try {
        if (conn != null) conn.close();
        scanner.close();
```

```
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

// select statement so we can see all of the current loans.
private static void viewLoans(Connection conn) throws SQLException {
    System.out.println("--- CURRENT LOANS ---");
    Statement stmt = conn.createStatement();
    String query = "SELECT Member.FullName, Book.Title, Loan.CheckoutDate " +
                   "FROM Loan JOIN Member ON Loan.MemberID = Member.MemberID " +
                   "JOIN Book ON Loan.BookID = Book.BookID";

    ResultSet rs = stmt.executeQuery(query);
    boolean hasData = false;
    while (rs.next()) {
        hasData = true;
        System.out.println("> " + rs.getString("FullName") + " borrowed '" +
                           rs.getString("Title") + "' on " + rs.getString("CheckoutDate"));
    }
    if (!hasData) System.out.println("No books are currently checked out.");
}

// Insert statement to add new members.
```

```
  private static void addMember(Connection conn, Scanner scanner) throws
SQLException {
    Statement stmt = conn.createStatement();
    System.out.print("Enter new Member ID (e.g., 00004): ");
    String id = scanner.nextLine();
    System.out.print("Enter Full Name: ");
    String name = scanner.nextLine();
    System.out.print("Enter Email Address: ");
    String email = scanner.nextLine();

    String query = "INSERT INTO Member (MemberID, FullName, Email) VALUES ('" +
            id + "', '" + name + "', '" + email + "')";
    try {
      if (stmt.executeUpdate(query) > 0) System.out.println("Success! New member
added.");
    } catch (SQLException e) {
      System.out.println("Error: Could not add member. ID or Email must be unique!");
    }
  }

  // update statement to update the email.
  private static void updateEmail(Connection conn, Scanner scanner) throws
SQLException {
    Statement stmt = conn.createStatement();
    System.out.print("Enter Member ID to update (e.g., 00001): ");
    String id = scanner.nextLine();
    System.out.print("Enter the new email address: ");
```

```
        String email = scanner.nextLine();

        String query = "UPDATE Member SET Email = '" + email + "' WHERE MemberID = '" + id +
"'";
        if (stmt.executeUpdate(query) > 0) {
            System.out.println("Success! Email updated.");
        } else {
            System.out.println("Error: Member ID not found.");
        }
    }

    // delete statement so they can return a book.
    private static void returnBook(Connection conn, Scanner scanner) throws SQLException
{
        Statement stmt = conn.createStatement();
        System.out.print("Enter Loan ID to process return (e.g., 00001): ");
        String id = scanner.nextLine();

        String query = "DELETE FROM Loan WHERE LoanID = '" + id + "'";
        if (stmt.executeUpdate(query) > 0) {
            System.out.println("Success! Book returned and loan record deleted.");
        } else {
            System.out.println("Error: Loan ID not found.");
        }
    }

    // Transaction so 2 people cant take the same book.
```

```
private static void checkoutBook(Connection conn, Scanner scanner) {
    System.out.print("Enter Member ID (e.g., 00001): ");
    String memberId = scanner.nextLine();

    System.out.print("Enter Book ID to checkout (e.g., 00001): ");
    String bookId = scanner.nextLine();

    System.out.print("Enter today's date (YYYY-MM-DD): ");
    String date = scanner.nextLine();

    // We create a random LoanID for this example (in a real app, this would auto-generate)
    String loanId = "L" + (int)(Math.random() * 10000);

    try {
        // 1. START THE TRANSACTION (Turn off auto-save)
        conn.setAutoCommit(false);
        Statement stmt = conn.createStatement();

        // 2. CHECK IF THE BOOK IS AVAILABLE
        String checkQuery = "SELECT * FROM Loan WHERE BookID = '" + bookId + "'";
        ResultSet rs = stmt.executeQuery(checkQuery);

        if (rs.next()) {
            // The book is already in the Loan table!
            System.out.println("Error: That book is currently checked out by someone else.");
```

```
            // Cancel the transaction
            conn.rollback();
        } else {
            // 3. THE BOOK IS FREE! Insert the loan record.
            String insertQuery = "INSERT INTO Loan (LoanID, BookID, MemberID,
CheckoutDate) VALUES ('" +
                        loanId + "', '" + bookId + "', '" + memberId + "', '" + date + "')";
            stmt.executeUpdate(insertQuery);

            // Save the transaction permanently
            conn.commit();
            System.out.println("Success! Book checked out to Member " + memberId);
        }

    } catch (SQLException e) {
        System.out.println("A database error occurred during checkout.");
        try {
            // If anything crashes, undo all changes
            conn.rollback();
        } catch (SQLException rollbackEx) {
            rollbackEx.printStackTrace();
        }
    } finally {
        try {
            // 4. ALWAYS TURN AUTO-COMMIT BACK ON when finished!
            conn.setAutoCommit(true);
```

```
        } catch (SQLException autoCommitEx) {
            autoCommitEx.printStackTrace();
        }
    }
  }
}
```

GitHub Repository:

https://github.com/Yuuri89/Databases-Final-Project-CS1103